

Low Complexity Neural Network Digital Predistortion of Wideband Power Amplifiers through Feature Selection

Cel Thys, Rodney Martinez Alonso, Ali H. Alsarraf, Dominique Schreurs, Sofie Pollin
WaveCoRE, Department of Electrical Engineering, Katholieke Universiteit Leuven, Belgium

Abstract—Due to the continuous increase in communication bandwidth and the use of highly efficient yet nonlinear power amplifiers, Digital Predistortion (DPD) algorithms are becoming increasingly complex. In particular, neural network (NN) based DPD approaches using Phase-Normalized NN architectures often incur substantially higher computational costs than widely deployed polynomial-based methods, such as the Memory Polynomial (MP) and Generalized Memory Polynomial (GMP) models. To bridge this gap between research performance and practical implementation, we propose a low-complexity Feature Selection NN DPD architecture. The proposed method employs an offline feature-engineering pipeline based on the Least Absolute Shrinkage and Selection Operator (LASSO) and the Minimum Redundancy Maximum Relevance (MRMR) algorithm to construct a compact and informative input representation. Using measured wideband FR3 power amplifier datasets that are publicly released with this work, we demonstrate up to 30% reduction in computational complexity while maintaining comparable linearization performance.

Index Terms—digital predistortion (DPD), power amplifier (PA), time-delay neural network (TDNN)

I. INTRODUCTION

The evolution toward sixth-generation (6G) wireless systems is driving the exploration of new spectrum allocations beyond the congested sub-6 GHz and millimeter-wave (mmWave) bands. The upper mid-band, also known as Frequency Range 3 (FR3) and spanning roughly 7–24 GHz, has emerged as a promising candidate for 6G due to its favorable balance between coverage and available bandwidth [1]. A key challenge in this band is power efficiency: to meet demanding link budgets at these frequencies, power amplifiers (PAs) must operate close to saturation, where nonlinear distortion is most pronounced. Digital Predistortion (DPD) is the standard technique to linearize PAs in the digital domain, allowing efficient operation without sacrificing spectral purity [2]. As signal bandwidths continue to grow, the memory effects of wideband PAs become increasingly significant, requiring more complex DPD models.

Traditionally, DPD has been implemented using truncated Volterra series, of which the Memory Polynomial (MP) and Generalized Memory Polynomial (GMP) [3] are the most widely deployed examples. These models are attractive in practice because they are linear in their parameters, admitting closed-form least-squares solutions with low complexity. However, they rely on basis functions that become highly correlated for high nonlinearity orders or high memory depths, limiting

the scalability of Volterra models [4], [5]. Indeed, for wideband signals with high peak-to-average power ratio (PAPR) and high modulation order, higher-order Volterra expansions are numerically ill-conditioned, capping the achievable linearization performance. Neural Network (NN) based DPD has been widely investigated as a means to overcome these limitations [6]. By learning the predistortion from data rather than relying on a polynomial basis, NN models can linearize complex PA behavior not captured by Volterra representations [5], [7]–[9]. The real-valued time-delay NN [4] and its derivatives, such as residual architectures [7] and the phase-normalized neural network (PNN) [8], have demonstrated state-of-the-art linearization performance.

Despite this performance advantage, the adoption of NN-based DPD in practical implementation remains limited. The core obstacle is computational complexity [6]: NN models capable of outperforming Volterra series typically require a significantly larger number of operations per sample, with implementations reported in the literature commonly requiring thousands of Floating-point-Operations (FLOPs) per IQ sample [6]. This stands in contrast to the few hundred FLOPs typical of a MP or GMP model [10], and poses a serious challenge for real-time deployment in resource-constrained digital frontend hardware.

In this work, we improve the complexity-performance trade-off of NN DPD models by targeting the input features. The main contribution is a feature engineering pipeline that combines the Least Absolute Shrinkage and Selection Operator (LASSO) regularization [11], [12] and Minimum Redundancy Maximum Relevance (MRMR) ranking [13] to construct a compact, maximally informative, and non-redundant input feature set from the same Volterra features used in GMP models. The resulting Feature Selection NN-DPD achieves an improved complexity-performance tradeoff, matching the linearization accuracy of a PNN while reducing its computational cost by up to 30%. We validate the approach using measurements on two 15 GHz devices with 100 MHz wideband signals. To support reproducible research and open benchmarking [9], the measured FR3 PA datasets used in this work are publicly available [14].

The outline of the work is as follows: in Section II, we provide an overview of the considered DPD system model and the metrics used for DPD performance evaluation. Section III describes the proposed feature engineering pipeline and the

feature-based NN architecture. The analysis of the performance complexity tradeoff for DPD algorithms is presented in Section IV, including measured linearization results. We end our work with the main conclusion in Section V.

II. SYSTEM MODEL

In this section, we introduce the DPD problem formulation and the key metrics used to compare the performance and complexity of DPD algorithms.

The operation of the DPD system is as follows: an over-sampled communication waveform $x(n)$ is predistorted by a nonlinear DPD model, denoted by f_{PD} , to counteract the power amplifier (PA) nonlinearity. After predistortion, the waveform $z(n)$ is sent to the RF frontend, including the PA, after which an RF coupler provides a feedback signal $y(n)$.

$$z(n) = f_{PD}\{x(n)\}, \quad (1)$$

$$y(n) = f_{PA}\{z(n)\}. \quad (2)$$

We implement an indirect learning approach to DPD training, where measured PA response data are used to identify an inverse model that maps the PA output back to its input [15], essentially performing post-distortion. Assuming that the PA behavior is sufficiently invertible within the operating region of interest, the identified post-distorter can be used as a predistorter by placing it before the PA. The predistortion function is implemented using truncated Volterra series or neural network models. Each model has coefficients θ , which are optimized during the identification (or training) stage to minimize the mean square error (MSE):

$$\min_{\theta} \frac{1}{N} \sum_{n=1}^N |x(n) - f_{PD,\theta}\{y(n)\}|^2 \quad (3)$$

This minimization problem is solved in closed form using least squares for Volterra-series models, whereas for NN-based models it is solved iteratively using stochastic gradient descent.

A. Measurement system

The setup for wideband modulated measurements [16], pictured in Fig. 1, consists of: a microwave vector signal generator (VXG) and a vector network analyzer (PNA-X) for signal generation and capture, bidirectional couplers, the Device-Under-Test (DUT), an attenuator (ATT) after the DUT to limit output power, and a 50-Ohm load.

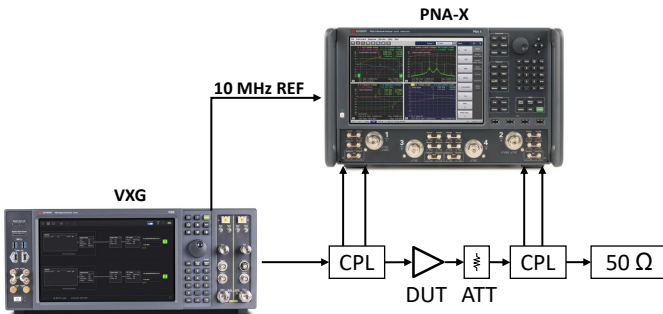


Fig. 1: Block diagram of the VNA-based measurement setup for DPD.

Each experiment employs a single-carrier input waveform with 100 MHz 64-QAM modulated data, root raised cosine pulse shaping with factor $\alpha = 0.35$ and sampling at 1.28 GHz. Each dataset is split into training, validation, and test captures with proportions of 33-33-33 %, respectively. DPD algorithm training and validation is performed in MATLAB, while the best-performing models are validated on the actual measurement setup. The measured FR3 PA response datasets are available at [14].

B. Performance metrics

After model identification, we use the normalized mean square error (NMSE) (4) on a validation dataset that was not used for model identification as the performance metric to compare predistorters. Here $x(n)$ represents the true PA input and $z(n)$ represents the NN model prediction:

$$NMSE = 10 \cdot \log_{10} \left\{ \frac{\frac{1}{N} \sum_{n=1}^N |z(n) - x(n)|^2}{\frac{1}{N} \sum_{n=1}^N |x(n)|^2} \right\}. \quad (4)$$

For the best-performing models, we measure the PA response to the DPD waveform and report the error vector magnitude (EVM) and the adjacent channel leakage ratio (ACLR). Note that the validation NMSE can be calculated for each trained model in simulation, while the EVM and ACLR are measurement metrics reported by the testbench equipment:

$$EVM = 100 \cdot \sqrt{\frac{\frac{1}{N} \sum_{n=1}^N |y(n) - x(n)|^2}{\frac{1}{N} \sum_{n=1}^N |x(n)|^2}}, \quad (5)$$

$$ACLR = 10 \cdot \log_{10} \left(\frac{P_{adj}}{P_{main}} \right). \quad (6)$$

In the equations above, $y(n)$, the output of the combined DPD-PA system, is normalized by the mean gain of the PA, while P_{adj} , P_{main} represent adjacent channel and main channel power, respectively. The measured EVM includes symbol recovery and equalization filtering as implemented in the VNA instrument.

C. DPD algorithmic complexity

In addition to the performance metrics mentioned above, we evaluate the DPD algorithmic complexity. Similar to [10], we focus on comparing the runtime complexity of DPD algorithms, as this is the most critical for real-time implementation. The complexity of offline model training or online adaptation is not considered, as this functionality does not need to run in real time.

Analysis of DPD complexity is typically limited to considering the number of real-valued parameters in the DPD model [3] as an indicator of its memory cost. We use FLOPs as a more accurate complexity metric and compare different DPD algorithms in an implementation-agnostic way [10]. In this

way, we compare the number of computations to be performed by a hardware platform to execute the DPD algorithm, without considering deeper implementation details such as parallelism or pipelining.

We introduce the symbol C for complexity in FLOPs [10], with a subscript indicating a specific algorithm; e.g., C_{MP} denotes the complexity of a Memory Polynomial DPD model. To calculate C for specific DPD models, we must identify all operations performed in the model and their associated FLOPs cost. For Volterra models, we follow the approach in [10], splitting the complexity cost into basis construction and filtering: $C = C_B + C_F$. For NN models, we develop our own approach by summing the costs of input feature generation, phase normalization preprocessing, the hyperbolic tangent nonlinear activation, and fully connected layer matrix multiplication and bias addition.

TABLE I: Computational complexity expressions in FLOPs/sample.

Model	Component	Complexity
MP	Basis generation	$C_{MP,B} = 3 + 7 + 2(P - 1)$
	Filtering	$C_{MP,F} = 8MP - 2$
	Total	$C_{MP} = C_{MP,B} + C_{MP,F}$
GMP	Basis generation	$C_{GMP,B} = 3 + 7 + 2(K_a - 1) + 4(K_b - 1)M_b$
	Filtering	$C_{GMP,F} = 8L_a(K_a + 2K_bM_b) - 2$
	Total	$C_{GMP} = C_{GMP,B} + C_{GMP,F}$
NN	Phase norm.	$C_{PN}(I) = 6(I + 1)$
	Fully connected	$C_{FC}(I, O) = 2IO$
	Tanh activation	$C_{TANH}(I) = 10I$

This work considers two Volterra series approaches: the MP model with memory depth M and polynomial order P ; and the GMP model with memory depth $L_a = L_b = L_c$, polynomial order K_a and $K_b = K_c$ and delay/advance memory $M_b = M_c$ (notation from [3]). For both models, we use even and odd-order envelope terms; although using only odd orders would achieve lower complexity, the DPD performance was measured to be similar. The cost of basis construction C_B accounts for the generation of envelope terms of the form $|x(n)|^p$, while discounting any terms that can be taken from a delay line. The cost of Volterra filtering C_F is derived from the number of model parameters, which is $M \cdot P$ for the MP model and $K_aL_a + 2K_bM_bL_a$ for the GMP model.

We consider two neural network approaches to DPD: the baseline PNN [8] and the proposed Feature Selection NN. Complexity formulations for all three are listed in Table I above, where I and O represent the size of that layer's input and output. We count 6 operations for each complex multiplication in the phase-normalization operation, which is applied to all input features and also to the output of the neural network [8]. In accordance with low-complexity implementations of the Tanh activation, we set the number of operations for this layer to 10 [17].

The tunable hyperparameters, including the number of neurons per layer and the number of hidden layers determine the total complexity of NN model inference. Another hyperparameter is k , the number of input features, as explained in Section III. This is the input to the first hidden layer of the NN model. To enable comparison across different

complexities, these model hyperparameters are tuned using Bayesian optimisation [18], and in Section IV we report the Pareto front of the best-performing models.

III. PROPOSED LOW COMPLEXITY NN-DPD

The proposed approach, summarized in Fig. 2, decouples the DPD problem into two stages: an offline feature selection pipeline and a low-complexity neural network suitable for real-time inference. We hypothesize that state-of-the-art NN DPD algorithms are generally quite complex because they perform data-driven feature extraction during each inference. Our proposed method decouples feature extraction from inference, thereby reducing computational load.

The offline feature selection stage constructs a rich Volterra-inspired feature basis space and then applies a feature engineering process to identify the most informative and least redundant subset of features. The online stage consists of a compact real-valued neural network that maps the selected features to the predistorted output. Since the feature selection is performed once during model design and is not subject to real-time deadlines, its computational cost is not included in the DPD runtime complexity model introduced in Section II-C, where only the top- k feature generation and NN model inference are counted.

In this section, we adopt the conventional machine learning notation, where $x(n)$ is the NN model input and $y(n)$ denotes the NN target output. This corresponds to the notation for a PA behavioral modeling task, while under the indirect learning framework [15] introduced in Section II, these signal roles are to be swapped such that the learned DPD model approximates the inverse PA characteristic.

A. Feature Generation

The starting point for the feature basis space is a truncated Volterra-like decomposition of the input signal $x(n)$. Because the neural network operates on real-valued inputs, the complex baseband signal is projected into its real and imaginary parts. For memory depth M and maximum nonlinearity order P , each input sample contributes the following feature types:

$$x_I(n - m), \quad x_Q(n - m) \quad (7)$$

$$|x(n - m)|^p \quad (8)$$

$$x_I(n - l)x_Q(n - m) \quad (9)$$

$$x_I(n - l)|x(n - m)|^p, \quad x_Q(n - l)|x(n - m)|^p \quad (10)$$

where $m, l \in \{0, \dots, M - 1\}$ and $p \in \{1, 3, \dots, P\}$. The delay line (7) captures the linear memory of the DUT, while the envelope terms (8) account for memoryless nonlinearity with odd order p . The cross terms (9) nonlinearly combine I/Q terms at different memory depths, while the terms (10) combine instantaneous samples with envelope terms, similar to the GMP structure [3]. Together, these span a feature space with expressive power similar to that of a large GMP model, with the important distinction that all features are real-valued scalars. Note that in a typical NN DPD model, terms such as (9), (10) are not used as an input, as their construction

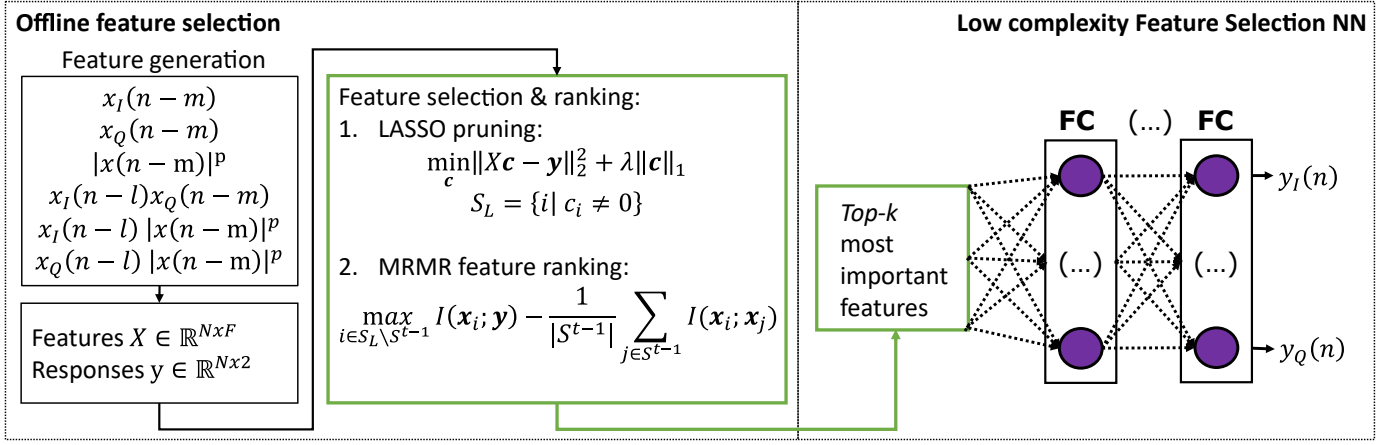


Fig. 2: Proposed feature selection approach to low complexity NN DPD.

would consume too much computational resources. The feature generation procedure yields a large input feature matrix $X \in \mathbb{R}^{N \times F}$, where the total number of candidate features F scales quadratically in M, P . The corresponding response matrix $y \in \mathbb{R}^{N \times 2}$ holds the real and imaginary parts of the target signal. In the proposed feature engineering pipeline, the number of training samples N is set to 15000, while $M = 200$ and $P = 7$, leading to $F = 321200$ features.

B. LASSO Feature Selection

A known limitation of high-order Volterra basis functions is their strong mutual correlation, which makes it difficult to assess individual feature relevance [2], [6]. While linear least-squares fitting in this setting is ill-conditioned, the LASSO (or ℓ_1 regularization) promotes sparsity and has relatively low complexity [12]. The LASSO objective combines the standard least squares formulation with an ℓ_1 sparsity penalty:

$$\hat{\mathbf{c}} = \arg \min_{\mathbf{c} \in \mathbb{R}^F} \|\mathbf{X}\mathbf{c} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{c}\|_1 \quad (11)$$

$$\mathcal{S}_L = \{i : \hat{c}_i \neq 0\}. \quad (12)$$

Here $\hat{\mathbf{c}}$ is the sparse coefficient vector of the LASSO model fit on the data and $\mathcal{S}_L$ is the subset of selected features. The regularization parameter λ is chosen via exhaustive search, aiming for approximately 2000 surviving features after the LASSO stage. This stage significantly reduces the candidate feature space before feature ranking.

C. MRMR Feature Ranking

The goal of this stage is to rank the selected features in $\mathcal{S}_L$ by order of importance, so that selecting the top k features as inputs for the NN will result in the most informative and least redundant features. We apply the MRMR [13] algorithm, which applies a greedy selection strategy, selecting in each iteration t one optimal feature i_t to add to the set $\mathcal{S}^t$. Starting from $\mathcal{S}^0 = \emptyset$, MRMR iterates as follows:

$$i_t = \arg \max_{i \in \mathcal{S}_L \setminus \mathcal{S}^{t-1}} I(\mathbf{x}_i; \mathbf{y}) - \frac{1}{|\mathcal{S}^{t-1}|} \sum_{j \in \mathcal{S}^{t-1}} I(\mathbf{x}_i; \mathbf{x}_j) \quad (13)$$

$$\mathcal{S}^t = \mathcal{S}^{t-1} \cup \{i_t\}. \quad (14)$$

where $I(\mathbf{x}_i; \mathbf{y})$ denotes the mutual information between the feature vector $\mathbf{x}_i$ and the output and $I(\mathbf{x}_i; \mathbf{x}_j)$ denotes the mutual information between features i and j . The greedy solution produces a ranked list i_t , and the top k features from this ranking are retained as the final input to the neural network. The hyperparameter k directly controls the complexity-performance tradeoff and is treated as a model hyperparameter subject to the Bayesian optimization described in Section II-C. The optimal range for k is 100-200 features.

D. Low-Complexity Feature Selection NN

We adopt the residual PNN architecture [7], [8] as the network architecture, replacing its input with the feature vector of size k , while keeping the phase-normalization operation for I and Q inputs. The network produces outputs $y_I(n)$ and $y_Q(n)$, from which the complex postdistorter signal is constructed. By using a small k relative to the full feature set F , the proposed pipeline can match or exceed the performance of an equivalently sized PNN while operating at a fraction of the computational cost.

IV. RESULTS

We analyze the complexity-performance tradeoff for the baseline DPD algorithms and the proposed Feature Selection NN, using measured data from two DUTs that we make available in [14]. To approximate the true performance-complexity tradeoff for each of the considered DPD models, we run a hyperparameter search using Bayesian optimisation [18] and plot the convex hull of hyperparameter combinations that result in optimal performance at minimal complexity. Finally, we validate the proposed feature selection approach on the measurement system and report the EVM and ACLR performance.

A. DPD modelling results

Our first analysis focuses on DPD performance on a 15GHz GaAs amplifier (ADL9006), which offers wideband amplification with a low gain of 15 dB. Fig. 3 shows the amplitude-amplitude (AM/AM) and amplitude-phase (AM/PM) response for this device at our chosen operation point of -2 dBm input power and 10 dB attenuation.

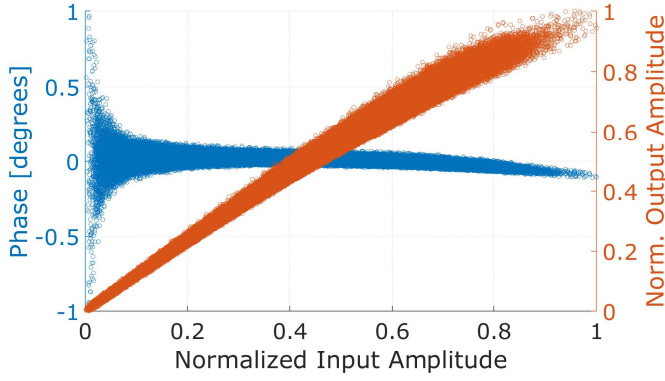


Fig. 3: Measured AM/AM and AM/PM response of the 1st DUT.

Fig. 4 shows the DPD validation NMSE of different algorithms, as a function of the memory footprint in terms of the number of real-valued parameters. It highlights the remarkable performance of NN-based DPD algorithms, with both the baseline PNN model and the proposed approach achieving an NMSE of -42 dB with 1600 parameters. The performance of the MP and GMP baselines stagnates quickly for relatively low number of coefficients.

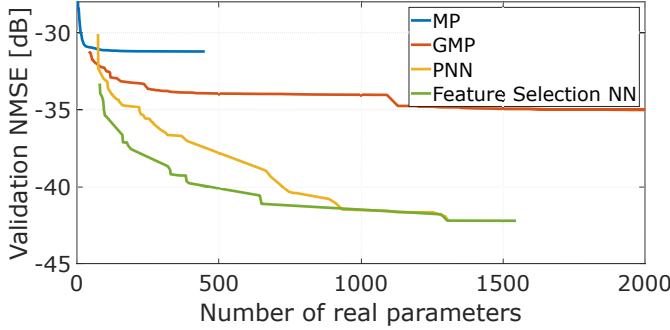


Fig. 4: Validation NMSE as a function of DPD model size.

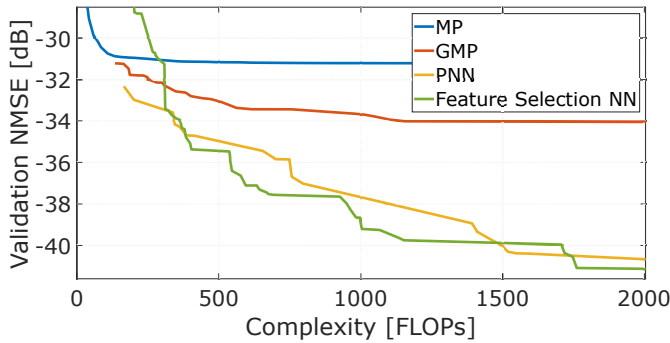


Fig. 5: Validation NMSE as a function of DPD computational complexity.

Fig. 5 shows the validation NMSE compared to DPD model computational complexity. It shows the complexity gap between the PNN baseline and the proposed Feature Selection NN. In fact, the proposed feature-based NN DPD model can significantly improve the tradeoff between NMSE and complexity, achieving -37 dB NMSE with only 595 FLOPs. For similar NMSE performance, the PNN baseline

requires 797 FLOPs, indicating the proposed approach offers an approximate 25% improvement in FLOPs.

Our next analysis concerns a high-gain PA (*TLPA2G22G-43-43-HS*). This PA promises gain up to 53 dB over the frequency range of 2 – 22 GHz, sacrificing linearity in the process. Fig. 6 shows the AM/AM and AM/PM responses for this device at -20 dBm input power and 30 dB output attenuation.

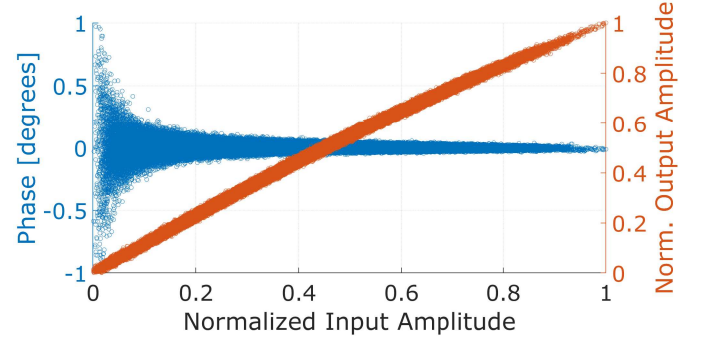


Fig. 6: Measured AM/AM and AM/PM response of the 2nd DUT.

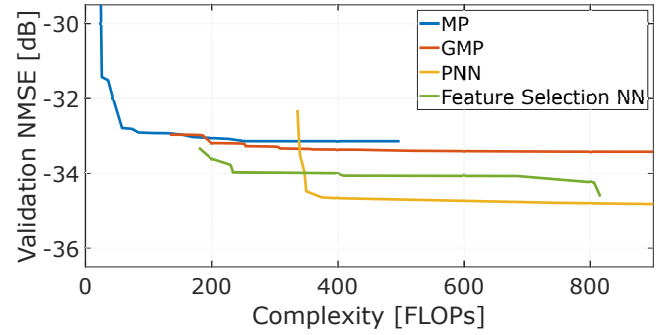


Fig. 7: Validation NMSE as a function of DPD computational complexity for the 2nd DUT.

Fig. 7 shows the validation NMSE as a function of complexity for this challenging linearization task. The proposed feature selection NN improves the complexity-NMSE tradeoff in the low complexity region, achieving -34 dB NMSE using only 234 FLOPs, which is a 32% reduction compared to the PNN baseline.

B. Measured DPD performance

Several candidate models covering the NMSE/FLOP tradeoff were evaluated experimentally by generating predistorted waveforms and measuring the DPD performance on the first DUT (*ADL9006*). The measured EVM and ACLR results are summarized in Table II, showing the FLOPs, the number of coefficients, and the performance metrics for 7 measured DPD models.

Table II shows that the proposed Feature Selection NN consistently achieves lower NMSE and EVM compared to a PNN model with the same complexity; for example it achieves 1.38% EVM with only 407 FLOPs, with the baseline PNN achieving 1.88%. The proposed Feature Selection NN achieves the lowest worst-case ACLR, up to 10dB lower than the

TABLE II: Measured DPD performance on the 1st DUT. ACLR is reported as the worst case (maximum of upper and lower adjacent channels).

Model	FLOPs	Coef.	NMSE [dB]	EVM [%]	ACLR [dBc]
No DPD	–	–	–	2.5	–33.0857
PNN	360	128	–32.7998	1.88	–39.9457
	787	287	–35.1755	1.54	–41.0505
	1839	725	–37.4431	1.39	–39.0286
	2491	911	–41.1065	1.39	–41.4168
Proposed	407	98	–35.3823	1.38	–43.8113
	939	344	–37.6726	1.24	–41.2444
	1768	650	–41.1615	1.08	–41.876

amplifier without DPD and 3.8 dB lower than a comparable PNN model.

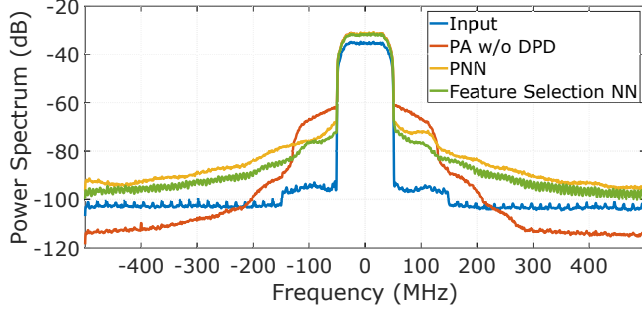


Fig. 8: Measured output spectrum after applying NN DPD algorithms with approximately 400 FLOPs.

Fig. 8 shows the measured spectrum after DPD linearization using the PNN and proposed models with approximately 400 FLOPs (the corresponding FLOP numbers are highlighted in Table II). The proposed feature selection NN approach achieves the lowest ACLR in this comparison. Finally, Fig. 9 shows the measured EVM as a function of model complexity, highlighting the excellent linearization capability of the proposed approach.

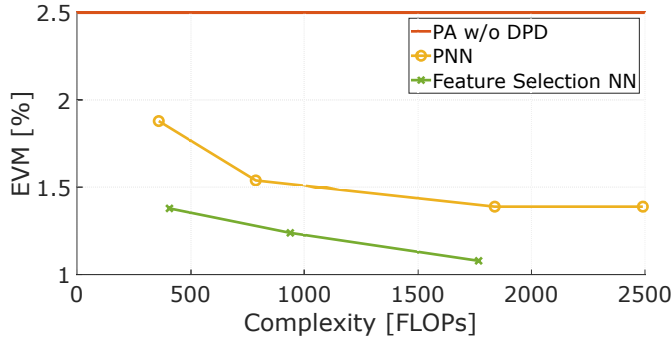


Fig. 9: Measured EVM of different DPD algorithms.

V. CONCLUSION

We proposed a Feature Selection Neural Network for low-complexity DPD operating in the FR3 frequency band, achieving an improved performance-complexity tradeoff compared to Volterra and Phase-normalized Neural Network baselines. Compared to the PNN baseline, the proposed approach reduces the computational cost by up to 30%. The method is validated by measured EVM and ACLR performance results. An interesting topic for further work is the inspection of the selected

features, possibly leading to more interpretable neural network DPD algorithms. The measured FR3 PA response datasets used to support this work are openly available at [14].

REFERENCES

- [1] Z. Cui, P. Zhang, and S. Pollin, “6G Wireless Communications in 7–24 GHz Band: Opportunities, Techniques, and Challenges,” in *2025 IEEE International Symposium on Dynamic Spectrum Access Networks (DySPAN)*, May 2025, pp. 1–8.
- [2] J. A. Becerra, K. S. Chuang, and P. L. Gilabert, “Digital Predistortion Linearization Demystified: Insights You Always Wanted to Know but Were Too Linear to Ask,” *IEEE Microwave Magazine*, pp. 2–21, 2026.
- [3] D. Morgan, Z. Ma, J. Kim, M. Zierdt, and J. Pastalan, “A Generalized Memory Polynomial Model for Digital Predistortion of RF Power Amplifiers,” *IEEE Transactions on Signal Processing*, vol. 54, no. 10, pp. 3852–3860, Oct. 2006.
- [4] T. Liu, S. Boumaiza, and F. Ghannouchi, “Dynamic behavioral modeling of 3G power amplifiers using real-valued time-delay neural networks,” *IEEE Transactions on Microwave Theory and Techniques*, vol. 52, no. 3, pp. 1025–1033, Mar. 2004.
- [5] R. Hongyo, Y. Egashira, T. M. Hone, and K. Yamaguchi, “Deep Neural Network-Based Digital Predistorter for Doherty Power Amplifiers,” *IEEE Microwave and Wireless Components Letters*, vol. 29, no. 2, pp. 146–148, Feb. 2019.
- [6] M. T. Khan, Y. Ding, and G. Goussetis, “Next-Gen Digital Predistortion From Hardware Acceleration of Neural Networks: Trends, Challenges, and Future,” *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–21, 2026.
- [7] Y. Wu, U. Gustavsson, A. G. i. Amat, and H. Wymeersch, “Residual Neural Networks for Digital Predistortion,” in *GLOBECOM 2020 - 2020 IEEE Global Communications Conference*, Dec. 2020, pp. 01–06.
- [8] A. Fischer-Bühner, L. Anttila, M. D. Gomony, and M. Valkama, “Phase-Normalized Neural Network for Linearization of RF Power Amplifiers,” *IEEE Microwave and Wireless Technology Letters*, 2023.
- [9] Y. Wu, G. D. Singh, M. Beikmirza, L. C. N. de Vreede, M. Alavi, and C. Gao, “OpenDPD: An Open-Source End-to-End Learning & Benchmarking Framework for Wideband Power Amplifier Modeling and Digital Pre-Distortion,” in *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*, May 2024, pp. 1–5.
- [10] A. S. Tehrani, H. Cao, S. Afsardoost, T. Eriksson, M. Isaksson, and C. Fager, “A comparative analysis of the complexity/accuracy tradeoff in power amplifier behavioral models,” *IEEE Transactions on Microwave Theory and Techniques*, vol. 58, no. 6, pp. 1510–1520, 2010.
- [11] D. Wisell, J. Jaldén, and P. Handel, “Behavioral Power Amplifier Modeling Using the LASSO,” in *2008 IEEE Instrumentation and Measurement Technology Conference*, May 2008, pp. 1864–1867.
- [12] A. Barry, W. Li, J. A. Becerra, and P. L. Gilabert, “Comparison of Feature Selection Techniques for Power Amplifier Behavioral Modeling and Digital Predistortion Linearization,” *Sensors*, vol. 21, no. 17, p. 5772, Jan. 2021.
- [13] H. Peng, F. Long, and C. Ding, “Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 8, pp. 1226–1238, Aug. 2005.
- [14] C. Thys, A. Alsarraf, R. Martinez Alonso, D. Schreurs, and S. Pollin, “FR3 PA response dataset for behavioral modelling and digital predistortion,” 2026, doi: 10.48804/306IIM.
- [15] C. Eun and E. Powers, “A new Volterra predistorter based on the indirect learning architecture,” *IEEE Transactions on Signal Processing*, vol. 45, no. 1, pp. 223–227, Jan. 1997.
- [16] J. P. Dunsmore and J.-P. Teyssier, “The Evolution of Vector Network Analyzers to Provide Precision Vector Spectrum Analysis for 6G Applications: VNA Evolves for 6G EVM Signals,” *IEEE Microwave Magazine*, vol. 25, no. 12, pp. 77–90, Dec. 2024.
- [17] N. H. F. Beebe, “Accurate hyperbolic tangent computation,” Center for Scientific Computing, Department of Mathematics, University of Utah, Salt Lake City, UT, USA, Tech. Rep., 1991. [Online]. Available: <https://www.math.utah.edu/~beebe/software/ieee/tanh.pdf>
- [18] J. Snoek, H. Larochelle, and R. P. Adams, “Practical Bayesian Optimization of Machine Learning Algorithms,” in *Advances in Neural Information Processing Systems*, vol. 25. Curran Associates, Inc., 2012.